



Hidden Camera Finder — PRD, Personas & Architecture



Planning doc for the iQOO Hyderabad City Battle, 26–27 Sept 2026.

Written before any real code, so the team agrees on what we are claiming, what we are not claiming, and what gets built in which order. The build sequence at the bottom is the one we follow under time pressure.

1. PRD



Problem

Hidden cameras in hotel rooms, guesthouses, PG accommodation, rental flats and retail trial rooms are a real and growing privacy violation in India. The people most at risk have no practical way to check a room. Professional detectors cost thousands of rupees and nobody carries one.

The free tools that do exist are mostly **RF scanners**, and they miss the case that matters most here: a cheap camera recording to an SD card with **no transmission at all**. They also give a black-box verdict with no way for the user to check the result.



What we are building

An offline mobile app that uses only the phone's existing sensors to point the user at **specific spots worth inspecting**, and always shows the raw signal behind the result.

✓ Goals

- Find candidate camera locations using **lens retroreflection** as the primary signal
- Use the **magnetometer as a pre-filter** to narrow down where to point the lens scan
- Work entirely offline — no network calls, no accounts, no cloud
- Make every verdict **independently verifiable** by the user
- Be usable by a non-technical person in an unfamiliar room in under two minutes

⊘ Non-goals



These are deliberate exclusions, not missing features. Saying them out loud is what makes the rest of the product defensible.

- **We do not certify a room as safe.** Absence of signal is not proof of absence.
- **We do not output a confidence percentage.** No “94% camera detected.”
- **We do not do RF detection.** Phones cannot scan arbitrary RF bands without extra hardware.
- **No cloud, no accounts, no analytics, no scan history upload.**
- We are not building a forensic or law-enforcement tool.



Core product principle



Show the evidence, not a score. Every flagged spot displays what fired and why — the magnetometer graph, or the captured frame with the blob circled. The user checks with their own eyes. This is the single biggest difference between us and every free app on the Play Store.

Requirements

P0 — must exist or there is no product

ID	Requirement
P0-1	Rear camera preview with torch on, processing a frame roughly every 300ms
P0-2	Bright-blob detection: grayscale, threshold to brightest 1–2%, filter for small and roughly circular
P0-3	Persistence tracking — count consecutive frames a blob survives
P0-4	Live overlay circling candidate blobs on the preview
P0-5	Session log of flagged spots with timestamps
P0-6	Results screen with RED / YELLOW / GREEN and raw evidence per spot
P0-7	Zero network calls — must run fully in airplane mode

P1 — strongly wanted

ID	Requirement
P1-1	Magnetometer sweep: rolling baseline, tunable spike threshold
P1-2	Live numeric readout plus animated needle or bar
P1-3	Rising-pitch tone or vibration as the spike strengthens
P1-4	Fusion logic combining glint and magnetic signals
P1-5	Copy framed as “metal object — worth a lens check”, never “camera detected”

P2 — conditional or nice to have

ID	Requirement
P2-1	Dark Room Scan for IR — only if the device test passes
P2-2	Guided room walkthrough checklist
P2-3	Save session locally

Verdict logic

Strong glint (held ≥ 3 frames)	-> RED	"Strong signal - inspect closely"
Glint + magnetic spike in same area	-> RED	"Strong signal - inspect closely"
Brief glint alone	-> YELLOW	"Possible lens reflection"
Magnetic spike alone	-> YELLOW	"Metal object - worth a lens check"
IR glow alone (if supported)	-> YELLOW	"Worth a lens check"
Nothing	-> GREEN	"Clear" (not "safe")



Why strong glint alone can reach RED: the most important case for this app is a cheap camera in a **plastic housing**, which produces almost no magnetic signature. If glint always needed magnetic confirmation, the app would systematically under-report exactly the threat it exists to catch. The magnetometer, by contrast, can **never** reach RED alone — ferrous metal is everywhere in a building and means nothing by itself.



Success criteria for the hackathon

- Plant a camera on stage and flag it in **under 60 seconds**
- Run the entire demo in **airplane mode**, visibly
- Every verdict traceable to a piece of raw evidence the judge can see
- Correctly say nothing when sweeping a clean surface — no false-positive theatre



Key risks

Risk	Mitigation
Magnetometer flags screws, brackets, wiring	Reframed as a pre-filter. It can never produce RED alone
IR feature may be impossible on this hardware	Device test in the first hour. Cut fast if it fails
Glint false positives from shiny surfaces	Persistence filtering — a lens holds its reflection, a sticker flashes once
"This already exists" objection	Concede it. Differentiate on sensor fusion and evidence transparency

Risk	Mitigation
Venue lighting ruins the glint demo	Test in the actual room. Carry a small dark box as a fallback prop

2. User personas

Priya, 26 — solo business traveller (primary)

- Stays in budget and mid-range hotels 6–8 nights a month, often booked last-minute.
- Checks the smoke detector and AC vent out of habit, but has no idea what she is actually looking for.
- **Needs:** a two-minute check on arrival, in an unfamiliar room, while tired.
- **Fears:** being told “all clear” and having it be wrong.
- **Design implication:** speed and honesty beat thoroughness. Never say “safe”.

Rahul, 22 — student in shared PG accommodation

- Lives in a room he did not furnish, with a landlord who has keys.
- Wants to check once, properly, then stop worrying.
- **Needs:** a systematic sweep, not a quick scan. Willing to spend 15 minutes.
- **Design implication:** the session log and results list matter more to him than speed.

Meera, 38 — boutique owner with customer trial rooms

- Legally and reputationally responsible for what happens in her trial rooms.
- Wants to check weekly and be able to say she checked.
- **Needs:** repeatability and a record.
- **Design implication:** drives the P2 local-save requirement. Not a hackathon priority.

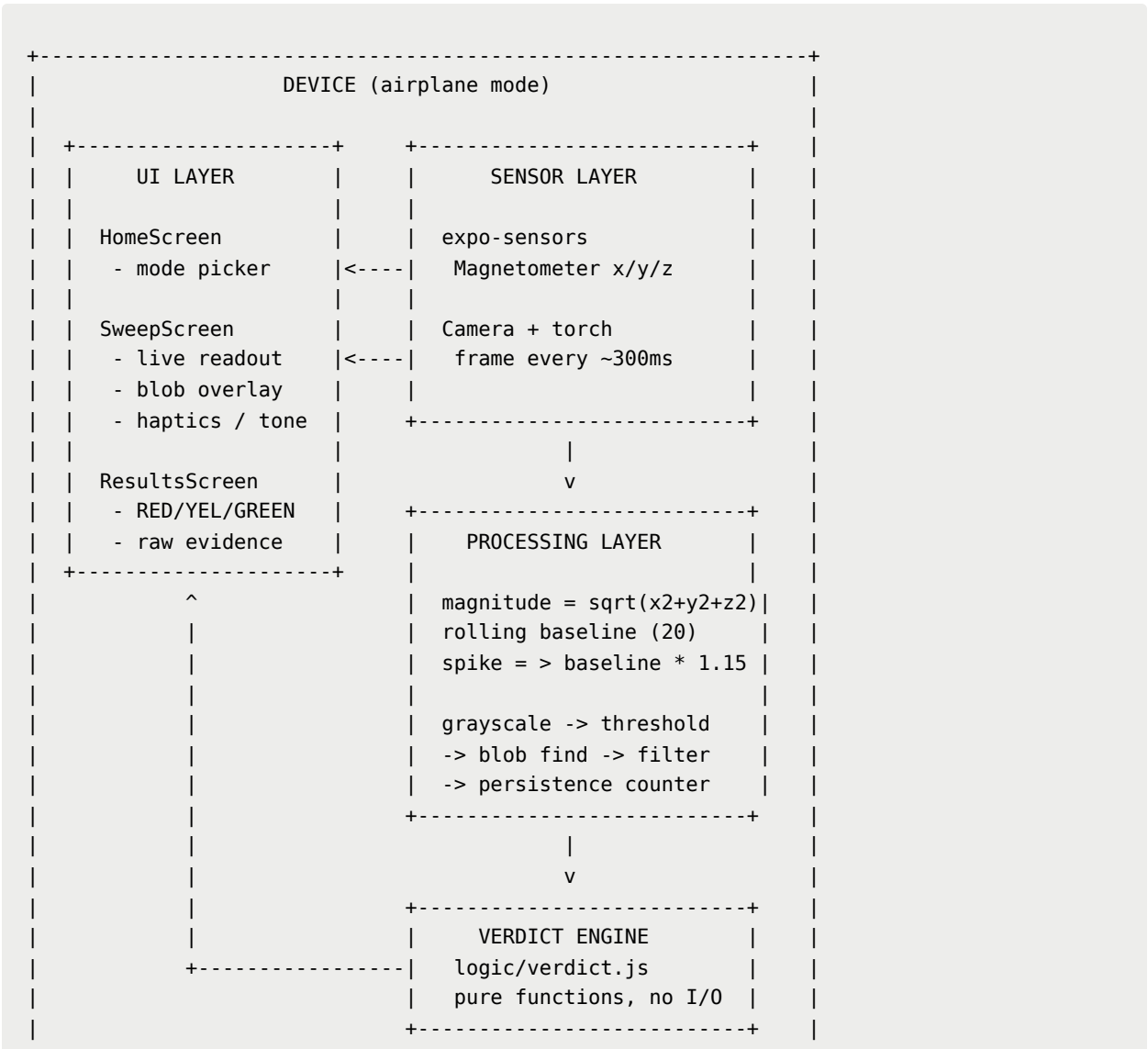
Anti-persona — the security professional

Someone wanting forensic certainty or RF spectrum analysis. **We do not serve this user.** Trying to would force us into accuracy claims we cannot support. If a judge pushes us here, the answer is that this is a consumer screening aid, not an inspection instrument.

3. Architecture

Prototype architecture (hackathon build)

Everything in one React Native process. No storage layer, no network layer, no native modules unless forced.



		Stage 3 temporal tracker: persistence + parallax across		
		frames as the user moves		
	+-----+			
	+-----+			
		SENSOR FUSION		
		Glint (primary) + magnetometer (filter) + IR (conditional)		
		Per-device calibration profile learned on first run		
	+-----+			
	+-----+			
		EVIDENCE STORE (encrypted, local only)		
		Scan sessions frames sensor traces		
		User-initiated export only. No background sync.		
	+-----+			
		Optional, opt-in, off by default:		
		- signed model updates (integrity-checked, no user data sent)		
		- anonymous false-positive reports, explicit consent each time		
	+-----+			



The privacy architecture is the product. An app that scans bedrooms and hotel rooms cannot have a cloud pipeline. Local-only is not a limitation we are working around — it is the reason the app is trustworthy, and it is why every arrow above stays inside the device boundary.

Prototype vs production — what actually changes

Layer	Prototype	Production
Frame processing	JS, ~3fps	Native worklet, 30fps
Classification	Threshold + shape rules	On-device model, bundled
Tracking	Consecutive-frame counter	Full temporal tracker with parallax
Calibration	Fixed constants	Per-device profile
Storage	Memory only	Encrypted local store
Network	None	None by default; opt-in signed model updates

4. Step-by-step implementation plan



Sequenced for risk, not for elegance. Step 0 can delete a feature before we spend an hour on it. Step 2 guarantees a demoable app exists early. The primary signal is built third on purpose — it is the hardest and most valuable, so it gets built once the safety net is in place.

Step 0 — Device tests (first hour, before any code)

1. **IR test.** Dark room, TV remote pointed at the front camera, press a button, look for a glow. Repeat on the rear camera. If neither sees it, **cut the Dark Room Scan immediately.**
2. **Glint test.** Torch on, point at a webcam or spare phone lens from 30cm to 2m. Confirm a bright stable point appears. Note the working distance, and note how a real lens holds its reflection while a sticker flashes once.
3. **Magnetometer baseline.** Walk a normal room, log the resting range and drift. Then hold near known metal. Set the threshold between the two.

Output: three recorded numbers and a go/no-go on the IR feature.

Step 1 — App shell (~1.5h)

Three screens with navigation, session state container, verdict engine as pure functions, mode definitions. No sensors yet. Shell-test buttons so the full data flow can be exercised before real signals exist.

Step 2 — Magnetometer sweep (~3h)

- Subscribe to the magnetometer, compute magnitude
- Rolling baseline over the last ~20 readings
- Spike when current exceeds baseline by the tuned threshold
- Live numeric readout plus animated bar
- Rising haptic pulse rate as the spike strengthens
- Log flagged spots with the sample window as evidence

Checkpoint: a demoable app now exists. Everything after this is upside.

Step 3 — Lens glint detection (~8h, the hard one)

- Camera preview with torch on
- Grab a frame every ~300ms
- Grayscale, threshold to the brightest 1–2%
- Find blobs, keep only small and roughly circular ones
- **Persistence counter** across consecutive frames
- Overlay circles on the preview
- Store the flagged frame as evidence

Budget the most time here and protect it. This is the primary signal and the entire pitch rests on it.

Step 4 — Fusion and results (~3h)

Wire both signals into the verdict engine, spatial and temporal correlation between a magnetic spike and a glint, results screen rendering the real evidence — graph for magnetometer, circled frame for glint.

Step 5 — Dark Room Scan (~2h, only if step 0 passed)

Reuse the step 3 pipeline on whichever camera sees IR. Prompt the user to kill the lights. If step 0 failed, skip entirely and say so honestly in the pitch.

Step 6 — Demo hardening (~4h, do not skip)

- Rehearse in the actual venue lighting
- Plant a test camera and time the detection
- Verify the whole flow in airplane mode
- Prepare the plastic-housing demo: magnetometer stays silent, glint still catches it
- Write the three prepared answers below into the deck

Prepared answers for the jury

"The magnetometer can't tell a camera from a screw."

Correct, and we do not claim it can. It is a cheap first-pass filter that tells the real scan where to look. The verdict is always glint-confirmed.

"This already exists on the Play Store."

RF detectors exist, and they miss the common case — a cheap camera recording to an SD card with nothing to transmit. We are not claiming to invent camera detection. We are fixing what the free tools get wrong: no signal fusion, and no way to verify the result yourself.

"How accurate is it?"

We deliberately do not publish an accuracy number, because we have not measured one on a representative sample and inventing one would be dishonest. That is exactly why every result shows the raw evidence instead of a score.

Build checklist

- ☐ Step 0 device tests done, three numbers recorded, IR go/no-go decided
- ☐ Step 1 shell running on the device
- ☐ Step 2 magnetometer sweep working — **demoable app secured**
- ☐ Step 3 glint detection working at a usable distance
- ☐ Step 4 fusion and evidence rendering
- ☐ Step 5 dark room scan, only if viable
- ☐ Step 6 rehearsed in venue lighting, airplane mode verified